Almacenamiento y recuperación de mensajes de buzón (VMS)

- **Versión:** 1.0 · **Fecha:** 26 de octubre de 2026
- **Estado:** borrador técnico para revisión. La parte "estado actual" describe la maqueta DEV; la parte "objetivo" es una **propuesta** no implementada.
- **Diagramas asociados:**
  - `arquitectura_alto_nivel.drawio` — página 1 *Vista lógica actual (DEV)* y página 2 *Vista objetivo producción (buzones)*.
  - `arquitectura_bajo_nivel.drawio` — página 1 *Despliegue y red DEV (puertos)*, página 2 *Secuencia depósito / MWI / recuperación (actual)* y página 3 *Almacenamiento objetivo (depósito y recuperación)*.
- **Documentos relacionados:** `HLD_Maqueta_IMS_uVAS.md` (HLD del entorno DEV) y `matriz_formatos_audio.md` (matriz de formatos de audio, en elaboración por separado; este documento no depende de su contenido).

> Convención de evidencias: [CODE] código/configuración versionada en el repo · [CONFIG] entorno/inventario · [DOC] documentación o bitácora · [INFERENCE] deducción técnica · [ASSUMPTION] supuesto que debe validarse o aprobarse.

1. Pregunta que responde este documento

¿Dónde se guardan los audios de los buzones de voz y cómo se recuperan, hoy en la maqueta y en una operación de varios cientos de miles de buzones con cientos —o cientos de miles— de mensajes por día? Los diagramas muestran ambas capas; aquí se justifican las decisiones y se dimensionan. [DOC]

2. Estado actual de la maqueta (DEV)

2.1 Dónde se guardan hoy los audios

Servicio	Cómo graba	Ruta dentro del contenedor `freeswitch`	Persistencia real
VMS simple `0100003`	`record ${vms_file} 300 0 0` en `public.xml`; nombre `msg_%Y%m%d_%H%M%S.wav`. [CODE]	`/var/lib/freeswitch/storage/vms/0100003/INBOX/` [CODE]	**Bind-mount** al host del nodo `.233`: `/home/administrator/maqueta-ims-3/freeswitch/vms` (`docker-compose.swarm.yml`). Sobrevive a la recreación de la tarea, pero solo en ese disco local. [CODE]
FEAT-02 depósito por MSISDN `00101xxxxx`	`voicemail default <dominio> <MSISDN>` (mod_voicemail). [CODE]	`storage-dir=/var/lib/freeswitch/storage/voicemail/default` → `<dominio>/<MSISDN>/msg_<uuid>.wav` (ruta que también usa `interfaz/app.py`, `VM2_BASE`). [CODE]	**No hay volumen ni bind-mount** para esa ruta: los mensajes viven en la capa escribible del contenedor. El índice de mod_voicemail (SQLite en `/var/lib/freeswitch/db`, ruta por defecto del paquete) tampoco está montado. Se pierden al recrear la tarea Swarm (`docker service update`, reubicación, nueva imagen). [CODE] [INFERENCE]
Prompts y flujos IVR	Ficheros versionados. [CODE]	`/mnt/freeswitch/audio/*.wav`, `/mnt/freeswitch/flows/*.json`. [CODE]	Bind-mount del directorio `freeswitch/` del repo. [CODE]
CDR de opción IVR	`ivr_flow.lua` añade una línea JSON por elección. [CODE]	`/mnt/freeswitch/logs/ivr_cdr.jsonl` [CODE]	Bind-mount. [CODE]

Formato: el perfil SIP `external` negocia solo PCMU/PCMA y tiene `disable-transcoding=true`; las grabaciones son WAV PCM16 lineal 8 kHz mono, ≈16 000 B/s (la UI calcula la duración como `(bytes − 44) / 16000`). Longitud máxima de grabación: 300 s (`record … 300`, `max-record-len=300`). [CODE]

Capacidad: cada nodo DEV tiene un disco de 30 GB según el inventario, y ese espacio lo comparten el sistema operativo, las imágenes y los logs. [CONFIG] Con 500 mensajes/día de 30 s en PCM16 (≈240 MB/día), y sin retención, el disco se llenaría en ≈4 meses aunque estuviera dedicado solo a audio. [INFERENCE]

2.2 Cómo se recuperan hoy

1. **Por voz (FEAT-02):** el UE llama a `0100004` y la iFC P30 lo encamina al AS. `vm_pin.lua` pide un PIN de 4 dígitos, lo valida con `GET http://mims3_ui:8888/api/pin/auth`, lista los mensajes con `vm_list`, los reproduce con `streamFile`, marca o borra con `vm_read` / `vm_delete` y publica el evento `MESSAGE_WAITING` que mod_sofia convierte en NOTIFY `message-summary`. [CODE] Secuencia completa en la página 2 del bajo nivel.
2. **Por web (UI):** `GET /api/vms/media` ejecuta `cat` dentro del contenedor `freeswitch` a través del proxy de la API Docker y devuelve el WAV. Esto solo funciona para los buzones de servicio (`0100002`, `0100003`). Para los buzones FEAT-02, la UI solo cuenta ficheros (`vm2_count`). [CODE]

2.3 Hallazgos del código relevantes para el diseño

#	Hallazgo	Evidencia	Impacto
H1	El almacenamiento de mod_voicemail (`storage/voicemail/default`) y su índice no están en ningún volumen.	`voicemail.conf.xml` + `volumes:` de `freeswitch` en el Compose Swarm. [CODE]	Pérdida de mensajes y del estado de MWI al recrear la tarea. Es la corrección mínima e inmediata para DEV (ver §8). [INFERENCE]
H2	`/api/vms/media` valida el nombre con `msg\d+\.wav`, pero los ficheros reales se llaman `msg_YYYYMMDD_HHMMSS.wav`.	`interfaz/app.py` y `public.xml`. [CODE]	El reproductor web responde 400 con las grabaciones reales. Habría que confirmarlo en DEV. [INFERENCE]
H3	El PIN se guarda en claro (`vas.subscriber_pin`). `vm_pin.lua` lo escribe en el log (`VMS_PIN attempt …`	`vm_pin.lua`, `interfaz/app.py`. [CODE]	No es aceptable en producción. El objetivo usa un hash Argon2id y no registra el PIN (§4, D9).

#	Hallazgo	Evidencia	Impacto
	`pin=`) y `/api/pin` lista todos los PIN sin autenticación.		[INFERENCE]
H4	La recuperación web pasa por `docker exec` (plano privilegiado) y no por un servicio de datos.	`vms_read_file()` en `app.py`. [CODE]	No escala y amplía la superficie del socket Docker (ver R4 del HLD). [INFERENCE]
H5	El AS solo admite G.711 y no transcodifica en llamada.	`external.xml`. [CODE]	Para almacenar en Opus hay que transcodificar fuera de la llamada. Un UE VoLTE con AMR-WB necesitaría además transcodificación en el borde o en el AS. [INFERENCE]

3. Supuestos de escala

Supuesto	Valor	Etiqueta
Buzones aprovisionados	300 000	[ASSUMPTION]
Mensajes depositados por día, escenario alto	500 000	[ASSUMPTION]
Mensajes depositados por día, escenario bajo ("cientos")	500	[ASSUMPTION]
Duración media de un mensaje	30 s	[ASSUMPTION]
Hora cargada	10 % del tráfico diario	[ASSUMPTION]
Escuchas por mensaje	1,5	[ASSUMPTION]
Retención de mensajes nuevos o leídos	30 días	[ASSUMPTION]
Mensajes que el usuario guarda (*saved*)	10 %, retenidos 90 días	[ASSUMPTION]
Tiempo de canal adicional por depósito (prompts, setup)	8 s	[ASSUMPTION]
Tiempo de canal por escucha (PIN, menú, audio)	45 s	[ASSUMPTION]

4. Arquitectura objetivo y decisiones

La vista general está en la página 2 del alto nivel y el detalle en la página 3 del bajo nivel. En los diagramas, los flujos se etiquetan D (depósito), R (recuperación por voz), W (recuperación web) y L (ciclo de vida). Todo lo que sigue es **propuesta** [INFERENCE], salvo donde se cita código.

D1 — Audio en almacenamiento de objetos S3-compatible, no en el disco del AS

- **Decisión:** guardar cada mensaje como un objeto inmutable en un bucket `vm-audio` de un almacenamiento S3-compatible on-premise (MinIO distribuido o Ceph RGW), con *erasure coding* 4+2 en el tier caliente. [INFERENCE]
- **Por qué:** 500 000 objetos/día de ~60 KB son ~15 M objetos en 30 días. Un sistema de objetos escala en capacidad y en número de objetos sin el límite de inodos ni el acoplamiento a un nodo que tienen el disco local o un NFS. Además, desacopla los servidores de medios del dato. [INFERENCE]
- **Alternativas descartadas:** el disco local del AS (estado actual) impide escalar o sustituir nodos. NFS/GlusterFS es un punto único de bloqueo con mucha metadata de directorios. Guardar BLOBs en la BD hace crecer los backups y la replicación de forma desproporcionada. [INFERENCE]

D2 — Esquema de claves particionado y sin datos personales en claro

```
{tenant}/{h2}/{msisdn_hmac}/{yyyy}/{mm}/{dd}/{msg_uuid}.opus
greetings/{tenant}/{msisdn_hmac}/greeting.opus
```

- `msisdn_hmac` = HMAC-SHA256(MSISDN, clave del KMS) en hexadecimal. El MSISDN no aparece en claves, logs de acceso ni URLs. [INFERENCE]
- `h2` = dos primeros caracteres hexadecimales del hash: reparten los prefijos en 256 particiones lógicas y evitan *hot prefixes*. [INFERENCE]
- La fecha en la clave permite reglas de lifecycle e inventarios por día. `msg_uuid` es el UUID de la llamada o de la grabación, lo que hace idempotentes los reintentos de subida. [INFERENCE]
- La clave **no** se usa para listar buzones: listar objetos S3 por prefijo no es una operación de baja latencia. El listado se hace siempre contra la BD de metadatos (D3). [INFERENCE]

D3 — Metadatos en PostgreSQL particionado con índice por buzón

- **Tabla `vm_message`**, particionada por rango mensual sobre `created_at` (ver §6). Índice `(mailbox_id, folder, created_at DESC)` sobre filas no borradas. Listar un buzón es una búsqueda por índice B-tree, O(log n), en 1-4 particiones activas (30 días de bandeja más hasta 90 de guardados). La paginación es por cursor `(created_at, msg_id)`. [INFERENCE]
- **Por qué PostgreSQL:** transacciones para los cambios de estado (nuevo → leído → borrado), `ON CONFLICT` para la idempotencia, particionado declarativo y `DROP PARTITION` barato para la retención. El volumen (~18 M filas, ~7 GB, §7) cabe en un primario con réplicas. [INFERENCE]
- **Alta disponibilidad:** un primario, una réplica síncrona en otra zona y una réplica asíncrona de lectura. Gestión con Patroni o un operador equivalente. [ASSUMPTION]

D4 — Contadores MWI en Redis como caché, con la verdad en PostgreSQL

- `HSET mwi:{mailbox_id} new <n> saved <m>`, actualizado por el worker tras confirmar el `INSERT` y por la VM API tras cada `PATCH`. [INFERENCE]
- Si la clave no existe o se detecta divergencia, se recalcula con `COUNT` sobre el índice de D3. Redis no necesita persistencia fuerte porque es reconstruible. 300 000 claves ocupan ~30 MB. [INFERENCE]
- En la maqueta, el equivalente es `vm_boxcount` de mod_voicemail. El Redis actual solo está configurado como auxiliar de PyHSS. [CODE]

D5 — Servidores de medios FreeSWITCH sin estado, con escritura local temporal y subida asíncrona

1. **D2 del diagrama:** FreeSWITCH graba en un *spool* local (SSD o tmpfs) `/var/spool/vm/<uuid>.wav` en PCM16 8 kHz, igual que hoy. [INFERENCE]
2. **D3-D4:** un agente local del nodo, que escucha `RECORD_STOP` o `CHANNEL_HANGUP` por ESL, transcodifica a Opus/Ogg y hace un `PUT` del objeto con `Content-MD5` e `If-None-Match: *`. Reintenta con *backoff* exponencial. Conserva el `.opus` en el spool hasta recibir el ACK y borra el WAV tras la transcodificación. [INFERENCE]
3. **D5-D9:** publica `vm.stored {msg_id, mailbox_id, key, size, duration, codec, created_at}` en una cola durable (Kafka, RabbitMQ *quorum queues* o Redis Streams), particionada por `mailbox_id` para conservar el orden por buzón. Los workers de metadatos hacen `INSERT … ON CONFLICT DO NOTHING`, `HINCRBY` en Redis y emiten `mwi.changed`. Los fallos van a una DLQ con alerta. [INFERENCE]
4. **D10:** el publicador MWI entrega el evento `MESSAGE_WAITING` al AS que mantiene la suscripción. AS-Front reparte SUBSCRIBE e INVITE con un hash por buzón para que la suscripción caiga en un nodo predecible. Si ese nodo cae, el UE se vuelve a suscribir: en la maqueta `force-subscription-expires=60` [CODE].
5. **Reconciliador diario:** vuelve a subir el contenido del spool sin ACK, re-publica los objetos que no tienen fila y alerta de las filas sin objeto. [INFERENCE]

Por qué asíncrono: la llamada termina sin esperar al almacenamiento central. Si cae S3 o la BD, los mensajes se acumulan en el spool (~12 GB por cada 4 h de corte en todo el pool en hora cargada, §7) y no se pierden. Al ser idempotente por `msg_id`, reintentar no duplica. [INFERENCE]

D6 — Recuperación por el canal de voz

1. **R1:** tras el PIN, FreeSWITCH llama a la VM API (`POST /mailboxes/{id}/pin:verify` y `GET /mailboxes/{id}/messages?folder=inbox`). [INFERENCE]
2. **R2:** la VM API lee del índice de D3, en la réplica de lectura para listar y en el primario para cambiar estado. [INFERENCE]
3. **R3-R4:** FreeSWITCH reproduce con `mod_http_cache` (`http_cache://<URL prefirmada>`), que mantiene una caché LRU local. Tras validar el PIN se precargan los primeros mensajes nuevos para que el primer audio no espere a la red. [ASSUMPTION] Hay que verificar que la imagen incluya `mod_http_cache` y `mod_opusfile` (lectura de `.opus`). Si no los incluye, el agente local puede descargar el objeto y entregar un WAV temporal.
4. El audio se decodifica a L16 y se codifica al códec de la llamada (G.711 PCMU/PCMA como hoy, o AMR-WB/EVS si se habilita en el borde). [INFERENCE]
5. **R5:** `PATCH` de estado (read, saved o deleted) en la VM API, que actualiza la BD y Redis y dispara MWI. Un borrado es lógico (`state=deleted`, `deleted_at`) y el objeto se purga después (D8). [INFERENCE]

D7 — Recuperación por API o web con URL prefirmada

- **W1-W2:** portal o app → API Gateway (OAuth2/OIDC, TLS, rate-limit, auditoría) → VM API, que comprueba la propiedad del buzón. [INFERENCE]
- **W3:** la VM API firma una URL GET de corta vida (TTL 300 s) sobre la clave exacta, sin mover audio por el backend. [INFERENCE]
- **W4:** el navegador descarga directamente del bucket con `Range`. Si el cliente no soporta Ogg/Opus, un endpoint de transcodificación bajo demanda entrega MP3 o AAC y cachea el resultado con un TTL corto. [ASSUMPTION]

D8 — Retención, ciclo de vida y tiering

Regla	Valor propuesto	Etiqueta
Nuevos y leídos no guardados	Expiran a los 30 días	[ASSUMPTION]
Guardados	90 días; transición a tier frío (HDD, EC 8+3) a los 30 días	[ASSUMPTION]
Borrado por el usuario	Lógico al instante; purga física del objeto a los 7 días	[ASSUMPTION]
Particiones de metadatos	`DROP PARTITION` del mes cuando todas sus filas expiraron (≈4 meses)	[INFERENCE]
Cuota por buzón	Máximo de mensajes o minutos por buzón; rechazo con aviso al llamante	[ASSUMPTION]

Las expiraciones las ejecuta un job que borra objetos y filas de forma coordinada (L3). La regla de lifecycle del bucket queda como red de seguridad: borra todo objeto con más de 100 días que no tenga *legal hold*. [INFERENCE] Cualquier retención por requisito regulatorio debe definirla negocio o legal. [ASSUMPTION]

D9 — Réplicas, backup y cifrado

- **Objetos:** EC dentro del sitio y replicación asíncrona del bucket a un sitio B (RPO de minutos). *Versioning* activado, con expiración de versiones antiguas a 7 días. *Object lock* opcional para retención legal. [ASSUMPTION]
- **Metadatos:** backup base semanal y archivado continuo de WAL a un bucket de backup (PITR). Restaurar significa volver a un punto en el tiempo y reconciliar con el inventario del bucket. [INFERENCE]
- **Cifrado:** SSE-KMS por tenant en reposo (KMS o Vault), TLS en todos los saltos (agente → S3, VM API → BD) y PIN almacenado con Argon2id más un *pepper* en el KMS. El PIN nunca aparece en logs ni en URLs; se bloquea tras 3 intentos fallidos. Esto corrige H3. [INFERENCE]
- **Accesos:** los nodos de medios solo pueden escribir en `vm-audio/*` (PUT) y leen mediante URL prefirmada. La VM API firma. Nadie salvo el job de ciclo de vida tiene permiso de borrado. [INFERENCE]

D10 — Formato de almacenamiento y de reproducción

- **Almacenamiento (tentativo):** Opus 16 kbit/s mono en contenedor Ogg, transcodificado por el agente fuera de la llamada. Reduce el espacio 8× frente al PCM16 actual (§7). La comparación completa está en `matriz_formatos_audio.md` (en elaboración). [ASSUMPTION]
- **Frecuencia de muestreo:** 16 kHz solo aporta calidad si la entrada es de banda ancha (AMR-WB o EVS en VoLTE). Con la entrada G.711 actual, el contenido es de banda estrecha (8 kHz) y Opus lo codifica sin pérdida de inteligibilidad a esa tasa.

[INFERENCE]
- **Reproducción hacia la red:** G.711 (PCMU/PCMA), como negocia hoy el AS, con transcodificación Opus → L16 → G.711 en FreeSWITCH al reproducir. [CODE] [INFERENCE]

5. Flujos objetivo resumidos

Paso	Origen → destino	Acción
D1	UE → IMS → AS-Front → FreeSWITCH	INVITE por desvío CDIV, igual que la iFC P40 actual [CODE]
D2	FreeSWITCH → spool	Graba WAV PCM16
D3-D4	Agente → S3	Transcodifica y hace PUT `…/{uuid}.opus`
D5-D6	Agente → cola → workers	`vm.stored`
D7-D8	Workers → PostgreSQL / Redis	INSERT idempotente y contador MWI
D9-D10	Workers → publicador MWI → AS	NOTIFY `Messages-Waiting: yes`
R1-R2	FreeSWITCH → VM API → PostgreSQL	PIN y listado por índice
R3-R4	FreeSWITCH → caché local → S3	GET prefirmado si falla la caché, y reproducción
R5	FreeSWITCH → VM API → PostgreSQL / Redis	read, saved o deleted, y MWI
W1-W4	Navegador → API GW → VM API; navegador → S3	URL prefirmada y descarga directa
L1-L5	Lifecycle, replicación, expiración, WAL, SSE	§4 D8-D9

6. Modelo de datos orientativo

```
CREATE TABLE vm_mailbox (
  mailbox_id   BIGINT PRIMARY KEY,
  tenant_id    SMALLINT NOT NULL,
  msisdn_hmac  BYTEA    NOT NULL UNIQUE,     -- nunca el MSISDN en claro fuera de la API de aprovisionamiento
  pin_hash     TEXT     NOT NULL,            -- Argon2id + pepper (KMS)
  pin_failures SMALLINT NOT NULL DEFAULT 0,
  quota_msgs   INT      NOT NULL DEFAULT 50,
  greeting_key TEXT
);

CREATE TABLE vm_message (
  msg_id      UUID        NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL,          -- instante de inicio de la grabación (determinista: idempotencia)
  mailbox_id  BIGINT      NOT NULL,
  tenant_id   SMALLINT    NOT NULL,
  folder      TEXT        NOT NULL CHECK (folder IN ('inbox','saved')),
  state       TEXT        NOT NULL CHECK (state IN ('new','read','deleted')),
  duration_ms INT         NOT NULL,
  codec       TEXT        NOT NULL,          -- p. ej. 'opus/16000/1'
  size_bytes  INT         NOT NULL,
  object_key  TEXT        NOT NULL,
  sha256      BYTEA,
  caller_id   TEXT,
  deleted_at  TIMESTAMPTZ,
  expires_at  TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (msg_id, created_at)
) PARTITION BY RANGE (created_at);

CREATE INDEX vm_message_mailbox_idx
  ON vm_message (mailbox_id, folder, created_at DESC)
  WHERE state <> 'deleted';
-- Particiones mensuales creadas por adelantado (pg_partman o job); DROP PARTITION al expirar.
```

[INFERENCE] La clave primaria de una tabla particionada debe incluir la columna de partición. Por eso la idempotencia se apoya en `(msg_id, created_at)`, y `created_at` viaja en el evento `vm.stored`.

7. Dimensionamiento orientativo

7.1 Fórmulas

- Bytes por mensaje: `B = tasa_bytes/s × duración + cabecera`.
  - PCM16 8 kHz: 16 000 B/s, más 44 B de cabecera WAV.
  - G.711 en WAV: 8 000 B/s.
  - Opus 16 kbit/s: 2 000 B/s, más ~1 KB de cabeceras y páginas Ogg. [INFERENCE]
- Volumen: `diario = N_msg/día × B` · `mensual = diario × 30` · `anual = diario × 365` (unidades decimales: 1 GB = 10^9 B).
- Volumen en régimen con retención (stock): `stock = diario × (30 + 0,10 × (90 − 30)) = diario × 36`. Corresponde a la bandeja de 30 días más el 10 % guardado hasta 90 días. [ASSUMPTION]
- Espacio físico: `stock × 1,5` (EC 4+2) `× 2` (sitio B con el mismo esquema) = `stock × 3`. [ASSUMPTION]

7.2 Resultados (duración media 30 s)

Formato	Tamaño por mensaje	500 msg/día: diario / anual	500 000 msg/día: diario	Mensual (30 d)	Anual (sin retención)	Stock con retención (36 d)	Físico (EC + sitio B)
WAV PCM16 8 kHz (actual)	480 KB	240 MB / 87,6 GB	240,0 GB	7,20 TB	87,6 TB	8,64 TB	25,9 TB
WAV G.711 8 kHz	240 KB	120 MB / 43,8 GB	120,0 GB	3,60 TB	43,8 TB	4,32 TB	13,0 TB
Opus 16 kbit/s mono, Ogg (propuesto)	≈61 KB	30 MB / 11,1 GB	30,5 GB	0,92 TB	11,1 TB	1,10 TB	3,3 TB

[INFERENCE] Valores calculados con los supuestos de §3. El escenario de 500 mensajes/día cabe en cualquier opción. Aun así, la maqueta actual no tiene retención, y su disco de 30 GB se agota en ≈4 meses con PCM16.

7.3 Magnitudes derivadas (escenario 500 000 mensajes/día)

Magnitud	Cálculo	Resultado
Depósitos en media	500 000 / 86 400 s	5,8 msg/s
Depósitos en hora cargada	500 000 × 10 % / 3 600 s	13,9 msg/s
Canales de depósito simultáneos	13,9 × (30 + 8) s	≈530 canales
Escuchas en hora cargada	500 000 × 1,5 × 10 % / 3 600 s	20,8 escuchas/s
Canales de escucha simultáneos	20,8 × 45 s	≈940 canales
RTP entrante de depósitos (G.711, 20 ms)	530 × 80 kbit/s	≈42 Mbit/s
Ingesta a S3 en hora cargada	13,9 × 61 KB (Opus) / × 480 KB (PCM16)	0,85 MB/s / 6,7 MB/s
Spool para 4 h de corte de S3 (todo el pool)	13,9 × 4 × 3 600 × B	12 GB (Opus) / 96 GB (PCM16)
Filas de metadatos en régimen	500 000 × 36	18 M filas, ≈7 GB con índices (~400 B/fila)
Memoria Redis MWI	300 000 × ~100 B	≈30 MB
Nodos FreeSWITCH	≈1 470 canales / 500 canales por nodo [ASSUMPTION]	3 nodos más 1 de reserva (N+1)

[ASSUMPTION] La capacidad por nodo (500 canales con reproducción desde Opus) debe medirse con una prueba de carga sobre el hardware real (ver R8 del HLD). Las tasas de PUT y GET resultantes (decenas por segundo) son bajas para MinIO o Ceph RGW. En este caso dimensionan la capacidad y la durabilidad, no el rendimiento. [INFERENCE]

8. Ruta de evolución desde la maqueta

1. **Inmediato en DEV, sin cambiar la arquitectura:** montar `/var/lib/freeswitch/storage/voicemail` y el índice de mod_voicemail en volúmenes del nodo `.233` (H1). Corregir la validación de `/api/vms/media` (H2). Dejar de registrar el PIN (H3). Este documento no aplica esos cambios a la maqueta. [INFERENCE]
2. **Piloto:** añadir un MinIO de un solo nodo y un agente de subida que copie los mensajes de mod_voicemail a S3 con el esquema de claves de D2, y una tabla de metadatos en el MySQL existente para medir la idempotencia y los tiempos. [ASSUMPTION]
3. **Producción:** VM API, PostgreSQL con HA, Redis para MWI, pool FreeSWITCH N+1 con spool y caché, MinIO o Ceph distribuido con EC y replicación a un segundo sitio, y políticas de §4 D8-D9 aprobadas. [ASSUMPTION]
4. **Validación:** ampliar `test_maqueta.sh` con un depósito que verifique el objeto en S3 y la fila en la BD, la recuperación desde una caché vacía y la caída de S3 durante un depósito (el spool debe retener el mensaje y el reconciliador recuperarlo). [ASSUMPTION]

9. Riesgos y pendientes

ID	Riesgo o pendiente	Acción
S1	Los mensajes de FEAT-02 no son persistentes en DEV (H1). [CODE]	Volúmenes para storage/voicemail y db antes de nuevas demos.
S2	Requisitos de retención, cuota y legales sin definir. [DOC]	Aprobar o sustituir los valores de §4 D8.
S3	`mod_http_cache` y `mod_opusfile` no verificados en la imagen FreeSWITCH. [ASSUMPTION]	Comprobar módulos o usar el agente como *fetcher* local.
S4	Afinidad de suscripciones MWI con un AS sin estado. [INFERENCE]	Hash por buzón en AS-Front y re-suscripción corta; probar la caída de un nodo.
S5	Compatibilidad de Ogg/Opus en navegadores antiguos. [ASSUMPTION]	Transcodificación bajo demanda a MP3 o AAC.
S6	Entrada AMR-WB/EVS en VoLTE frente al AS solo G.711. [CODE] [INFERENCE]	Decidir el punto de transcodificación (borde o AS) y su coste de CPU.
S7	PIN en claro y expuesto (H3). [CODE]	Hash, sin logs, API de aprovisionamiento autenticada.

10. Fuentes

- `maqueta-ims-3/freeswitch/conf/dialplan/public.xml` — extensiones 0100002/0100003/00101xxxxx/0100004. [CODE]
- `maqueta-ims-3/freeswitch/conf/autoload_configs/voicemail.conf.xml` — `storage-dir`, `max-record-len`, formato WAV. [CODE]
- `maqueta-ims-3/freeswitch/conf/sip_profiles/external.xml` — códecs, `disable-transcoding`, MWI. [CODE]
- `maqueta-ims-3/freeswitch/scripts/vm_pin.lua` e `ivr_flow.lua` — PIN, menú, MWI y CDR. [CODE]
- `maqueta-ims-3/freeswitch/init.sh` — almacenamiento VMS y `local_ip_v4` en Swarm. [CODE]
- `maqueta-ims-3/docker-compose.swarm.yml`, `docker-compose.yml`, `.env.swarm` — volúmenes, placement, puertos. [CODE] [CONFIG]
- `maqueta-ims-3/interfaz/app.py` — `/api/vms`, `/api/vms/media`, `/api/pin/auth`, `vas.subscriber_pin`. [CODE]
- `maqueta-ims-3/pyhss/maqueta_ifc.xml` — iFC P10/P30/P40. [CODE]
- `entregables-feat-02/02_feat02/FEAT-02_VMS_IMPLEMENTACION.md` — flujos FEAT-02. [DOC]
- `doc/hacom - maqueta IMS v1.1 - 10102026.csv` — nodos, CPU, RAM, disco y componentes (sin credenciales). [CONFIG]
- `entregables-feat-02/03_iteraciones/BITACORA_ITERACIONES.md` — MTU 1450 y fragmentación (G3). [DOC]